CHAPTER 10

FUNCTIONS

CHAPTER ESSENCE AT A GLANCE:
Master modular functional abstraction encompassing prototype declarations, formal parameter passing by value and reference, variable scope lifecycles, and stack-driven recursion.

4 CORE PRINCIPLES TO UNDERSTAND FIRST

• **Modular Functional Decomposition:** Dividing complex computational algorithms into cohesive, self-contained functions enhances software readability, reusability, and testing.

• **Call by Value versus Reference Binding:** Call by value copies argument values protecting original caller variables, whereas call by reference passes actual memory aliases allowing direct modification.

• **Scope and Storage Lifetime:** Local variables exist only within the execution of their declaring block stack frame, whereas global variables persist throughout entire program execution.

• **Base Case Recursive Unwinding:** Recursive functions must define a non-recursive terminating base condition to prevent runaway execution from causing call stack overflow.

ESSENTIAL ARCHITECTURE & CORE SYNTAX CARDS

Call by Value: void swap($int\, a, int\, b$) → Copies values

Pass by Value (Formal parameter isolation)

Call by Ref: void swap($int\, \&a, int\, \&b$) → Modifies actuals

Pass by Reference (Memory alias binding)

Prototype: $\mathrm{ReturnType\ FunctionName(ParamType_1,\ \ldots\ );}$

Function Declaration Signature Model

Recursion: $T(n) = T(n-1) + O(1)$ (Terminates at Base Condition)

Stack-Driven Recursive Unwinding

CRUCIAL EXAM PRECAUTIONS & VALUATION RUBRIC SECRETS

• **Alert:** Call by reference requires an ampersand (&) in the formal parameter declaration: void swap(int &a, int &b).

• **Alert:** Missing base condition in a recursive function causes infinite recursion, leading to stack overflow and catastrophic runtime crash.

• **Alert:** Functions with a return type other than void must explicitly execute a return statement along every possible execution path.

HIGH-YIELD CENTUM QUESTIONS & STEP-BY-STEP EXPLANATIONS

1. What is a function prototype in C++? Why is it necessary? [Score: 2 Marks]

Answer & Key Valuation Steps:

• **Function Prototype Definition:** A declaration statement specifying the function name, return type, and parameter types before its actual definition or invocation. **[1 Mark]**

• **Necessity:** Informs the compiler about the function signature so the compiler can validate argument types and counts during calls before full definition is reached. **[1 Mark]**

2. Distinguish between actual parameters and formal parameters with an example. [Score: 2 Marks]

Answer & Key Valuation Steps:

• **Actual Parameters:** The actual values or variables passed to the function in the function call statement (e.g. swap(x, y);). **[1 Mark]**

• **Formal Parameters:** The variables declared in the function header definition that receive the values or references from caller arguments (e.g. void swap(int a, int b)). **[1 Mark]**

3. Compare Call by Value and Call by Reference parameter passing methods with code illustrations. [Score: 3 Marks]

Answer & Key Valuation Steps:

• **Call by Value:** Values of actual parameters are copied into formal parameters. Changes made to formal parameters inside the function do not affect the original variables in the caller. **[1.5 Marks]**

• **Call by Reference:** Reference parameters (&) act as aliases to the original variables. Any modification made to formal parameters directly alters the actual caller variables in memory. **[1.5 Marks]**

4. Explain Local Variables and Global Variables on scope, lifetime, and storage location. [Score: 3 Marks]

Answer & Key Valuation Steps:

• **Scope:** Local variables are visible only within the declaring block or function; Global variables are declared outside all functions and accessible throughout the entire program. **[1 Mark]**

• **Lifetime:** Local variables are created on the stack upon block entry and destroyed upon exit; Global variables persist in static data memory for the entire program execution. **[1 Mark]**

• **Default Values:** Local variables contain unpredictable garbage values if uninitialized; Global variables are automatically initialized to zero. **[1 Mark]**

5. What is Recursion? State the two essential components required in every recursive function. [Score: 3 Marks]

Answer & Key Valuation Steps:

• **Definition:** A programming technique where a function calls itself directly or indirectly to solve a smaller instance of the same problem. **[1 Mark]**

• **Base Condition:** The terminating condition that stops recursion without making further recursive calls, preventing infinite loops. **[1 Mark]**

• **Recursive Step:** The statement where the function invokes itself with updated arguments progressively approaching the base case. **[1 Mark]**

6. Write a C++ function to swap two integer variables using Call by Reference and demonstrate its call from main(). [Score: 3 Marks]

Answer & Key Valuation Steps:

• **Function Definition:** void swapValues(int &a, int &b) { int temp = a; a = b; b = temp; } using reference parameters. **[1.5 Marks]**

• **Main Call & Output:** int main() { int x = 10, y = 20; swapValues(x, y); cout << x << ' ' << y; return 0; } Displays: 20 10. **[1 Mark]**

• **Boxed Result:** Direct memory modification verified through reference aliases. **[0.5 Mark]**

7. Write a recursive C++ function to calculate the factorial of a positive integer N. [Score: 3 Marks]

Answer & Key Valuation Steps:

• **Recursive Function Definition:** long long factorial(int n) { if (n <= 1) return 1; return n * factorial(n - 1); } **[1.5 Marks]**

• **Trace for N=4:** fact(4) = 4 * fact(3) = 4 * (3 * fact(2)) = 4 * 3 * (2 * fact(1)) = 4 * 3 * 2 * 1 = 24. **[1 Mark]**

• **Boxed Result:** Terminating base case returns 1, unwinding call stack correctly. **[0.5 Mark]**